

RoadMap AmaniERP

1. Diagnóstico do estado atual

1.1 Backend (`src/`) — maduro e à frente do frontend

Arquitetura confirmada: Clean Architecture + DDD Lite, .NET 8, EF Core (Npgsql) + Fluent API, Repository Pattern, MediatR (Commands/Queries), DTOs explícitos sem AutoMapper. A regra constitucional **estoque por movimentações** está respeitada (não há campo fixo de saldo; `EstoqueConsultaRepository.ObterSaldoAsync` calcula entradas – saídas).

Endpoints já implementados e funcionais:

Módulo	Endpoints	Situação
Produtos	<code>POST</code> , <code>GET</code> , <code>GET/{id}</code> , <code>PUT/{id}</code>	OK (sem saldo no DTO, por design)
Categorias (produto)	CRUD	OK
Clientes	CRUD + <code>inativar</code>	OK
Fornecedores	CRUD	OK
Compras	criar, obter, <code>recebimentos</code> , <code>perdas</code> , <code>em-transito</code> , <code>produtos-pendentes</code> , listar	OK (recebimento parcial + perdas + transação)
Vendas	criar (valida estoque + calcula lucro por custo médio), obter, listar, <code>dashboard</code> , <code>cancelar</code>	OK
Despesas	criar, listar	Parcial (sem update/delete; ver lacuna G2)
Contas a Receber	criar, <code>pagamentos</code> , listar, <code>por-cliente</code> , detalhe por cliente, update, delete	OK
Implantação	<code>inventario-inicial</code> , <code>saldo-inicial-caixa</code> , <code>contas-receber-iniciais</code>	OK
Dashboard Gerencial	consolidado, <code>financeiro</code> , <code>operacional</code> , <code>rankings</code> , <code>alertas</code> , <code>graficos</code>	OK (consultas agregadas via repositórios de leitura)
Dashboard Financeiro	<code>GET</code>	OK

1.2 Frontend (`frontend/`) — Next.js 15 / React 19 / Tailwind, Dark Only, Mobile First

- **Implementado de verdade:** Produtos (lista, novo, detalhe, editar) e Clientes (lista, novo, detalhe, editar, inativar). Design System completo (`app-shell`, `desktop-sidebar`, `mobile-bottom-nav`, estados loading/error/empty, UI base, TanStack Query).
- **Service pronto sem tela:** `services/suppliers.ts` existe, mas não há página de Fornecedores.
- **Apenas placeholders (`EmptyState`):** Dashboard (home), Compras, Vendas, Estoque, Financeiro, Configurações.

1.3 Features concluídas (specs 001–007)

001 Cadastros base · 002 Implantação inicial · 003 Mercadorias em trânsito/recebimento parcial · 004 Dashboards gerenciais · 005 Frontend base (Design System) · 006 Produtos (frontend) · 007 Clientes (frontend). **Todas backend-completas; do frontend só 006 e 007 entregaram telas operacionais.**

1.4 A lacuna central do MVP

O backend já consegue operar o negócio inteiro (comprar → receber → estoque → vender → financeiro → dashboard), mas o frontend só expõe cadastro de produtos e clientes. O caminho para o MVP é, majoritariamente, superfície de frontend sobre endpoints que já existem, mais alguns pequenos preenchimentos de lacuna no backend.

1.5 Lacunas técnicas detectadas no backend (confirmadas em código)

- **G1 — Consulta de estoque não exposta:** `IEstoqueConsultaRepository` só é usado internamente por `VendaService`. Não há controller para ler saldo nem histórico de movimentações. Bloqueia a tela de Estoque e o saldo no detalhe do produto.
- **G2 — Categorias de despesa sem endpoint:** `CriarDespesaCommand` exige `CategoriaDespesaId`, mas existe entidade `CategoriaDespesa` sem nenhum controller de criar/listar. Bloqueia o lançamento de despesas pelo frontend.
- **G3 — Sem autenticação/autorização** em toda a stack.
- **G4 (menor):** Despesas sem update/delete; Produtos sem inativar/excluir.

2. Features propostas (até o MVP)

| Numeração seguindo o padrão Spec Kit (a última concluída é 007).

F008 — Consulta de Estoque (backend)

- **Objetivo:** Expor, via API agregada, o saldo atual por produto e o histórico de movimentações, sem violar "estoque por movimentações".
- **Escopo exato:** Novo `EstoqueController` consumindo um read repository especializado que agrega `EstoqueMovimentacao` (entradas – saídas) com filtros.
- **O que entra:** `GET /api/estoque` (lista produtos com saldo calculado); `GET /api/estoque/{produtoId}/movimentacoes` (histórico com tipo, quantidade, origem compra/venda, custo, data); query agregada com filtros/limite; DTOs de resposta.
- **O que fica fora:** Ajuste manual de saldo, reserva, multi-depósito, transferências, inventário cíclico.
- **Dependências:** Nenhuma (entidades e `IEstoqueConsultaRepository` já existem).
- **Prioridade:** Alta (fundação para F012 e refinamentos de Produto).
- **Risco técnico:** Baixo. Atenção a performance (agregação no banco, não em memória — Princípio VII).
- **Ordem sugerida:** 1º.
- **Critérios de aceite:** Saldo calculado = entradas – saídas; nenhum campo fixo de saldo introduzido; consulta agregada com filtros; resposta via DTO; controller sem regra de negócio; build/migrations inalteradas (sem nova migration).
- **Impacto:** `src/.../Api/Controllers/EstoqueController.cs` (novo), `Application/Queries` + `Handlers` (novo), `Application/DTOs` (novo), `Ir` repo especializado), `Infra.IoC/DependencyInjection.cs`.

F009 — Gestão de Fornecedores (frontend)

- **Objetivo:** Permitir cadastrar/editar/consultar fornecedores, pré-requisito para Compras.
- **Escopo exato:** Módulo de frontend espelhando o padrão de Produtos (006), consumindo endpoints já existentes; `services/suppliers.ts` já existe.
- **O que entra:** Rotas `/fornecedores`, `/fornecedores/novo`, `/fornecedores/[id]`, `/fornecedores/[id]/editar`; lista com busca local; formulário; estados; item de navegação.
- **O que fica fora:** Histórico de compras por fornecedor, métricas, inativação (backend não tem `inativar` de fornecedor).
- **Dependências:** Nenhuma (backend pronto).
- **Prioridade:** Alta.
- **Risco técnico:** Muito baixo (replica 006).
- **Ordem sugerida:** 2º.
- **Critérios de aceite:** CRUD funcional contra `/api/fornecedores`; responsivo (mobile/tablet/desktop); Dark Only; `lint` / `typecheck` / `build` ok; sem cálculo de regra no frontend.

- **Impacto:** `frontend/src/app/fornecedores/**` (novo), `components/fornecedores/**` (novo), `hooks/use-suppliers.ts` (estender), `services/suppliers.ts`, `types/`, `config/navigation.ts` + `routes.ts`.

F010 — Implantação Inicial (frontend)

- **Objetivo:** Tela guiada para semear o sistema com dados reais da Amani: inventário inicial, saldo inicial de caixa e contas a receber iniciais — passo zero para o ERP refletir a realidade.
- **Escopo exato:** Fluxo (wizard/abas) em Configurações consumindo `POST /api/implantacao/*`.
- **O que entra:** Inventário inicial (entrada rastreável de estoque por produto); saldo inicial de caixa; contas a receber iniciais por cliente; confirmação e feedback de resultado.
- **O que fica fora:** Reabertura/edição em massa de implantação, importação de planilha (pós-MVP), recálculo de custo médio no frontend.
- **Dependências:** F009 (fornecedor/produtos) recomendável; produtos e clientes já existem.
- **Prioridade:** Alta (sem dados iniciais, estoque/financeiro não têm valor real).
- **Risco técnico:** Baixo-médio (UX de evitar duplo lançamento de inventário).
- **Ordem sugerida:** 3º.
- **Critérios de aceite:** Inventário inicial gera movimentação de entrada rastreável; caixa e contas iniciais registrados; mensagens de erro do backend exibidas; nenhuma regra recalculada no cliente; responsivo.
- **Impacto:** `frontend/src/app/configuracoes/**` ou `app/implantacao/**` (novo), `components/implantacao/**` (novo), `services/` + `hooks/` + navegação.

F011 — Compras e Recebimentos (frontend)

- **Objetivo:** Operar o ciclo de compra: registrar compra (mercadoria em trânsito), acompanhar pendências e registrar recebimento parcial e perdas — coração da entrada de estoque.
- **Escopo exato:** Frontend sobre `CompraController` (criar, `em-transito`, `produtos-pendentes`, `recebimentos`, `perdas`, listar).
- **O que entra:** `/compras` (lista + filtros de data/fornecedor), `/compras/nova` (itens, custo unitário, desconto/acréscimo), `/compras/[id]` (detalhe + trânsito), ação de **registrar recebimento** (parcial) e **registrar perda** (motivo Perda/Extravio/Avaria), visão de produtos pendentes.
- **O que fica fora:** Cancelamento de compra (não há endpoint), edição de compra, geração automática de estoque na criação (constitucionalmente proibido — só recebimento gera entrada).
- **Dependências:** F009 (fornecedores). Recebimento alimenta F012 (estoque).
- **Prioridade:** Alta.
- **Risco técnico:** Médio (UX de recebimento parcial; respeitar quantidade pendente; sem recálculo no cliente).
- **Ordem sugerida:** 4º.
- **Critérios de aceite:** Compra criada fica em trânsito sem gerar estoque; recebimento confirmado gera entrada; perda não gera estoque; quantidades validadas pelo backend; erros exibidos; responsivo.
- **Impacto:** `frontend/src/app/compras/**` (substituir placeholder), `components/compras/**` (novo), `services/purchases.ts` + `hooks/use-purchases.ts` + `types/` (novos), navegação.

F012 — Estoque (frontend)

- **Objetivo:** Visualizar saldo atual e histórico de movimentações por produto, além de itens pendentes de recebimento.
- **Escopo exato:** Frontend sobre F008 (`/api/estoque`) e `produtos-pendentes`.

- **O que entra:** `/estoque` (lista de produtos com saldo, busca, filtro), detalhe de movimentações por produto (entradas/saídas/origem), aba/visão de pendentes de recebimento.
- **O que fica fora:** Ajuste manual de saldo, transferência, alerta de estoque mínimo (pode entrar via dashboard), edição de movimentações.
- **Dependências:** **F008** (obrigatória), F011 (gera movimentações reais).
- **Prioridade:** Alta.
- **Risco técnico:** Baixo (somente leitura).
- **Ordem sugerida:** 5º (após F008/F011).
- **Critérios de aceite:** Saldo exibido vem do backend (sem cálculo no cliente); histórico mostra origem; estados loading/erro/vazio; responsivo.
- **Impacto:** `frontend/src/app/estoque/**` (substituir placeholder), `components/estoque/**` (novo), `services/stock.ts` + `hooks/` + `types/` (novos).

F013 — Vendas (frontend)

- **Objetivo:** Registrar vendas com validação de estoque e visualização de lucro (custo médio), listar e cancelar.
- **Escopo exato:** Frontend sobre `VendasController` (criar, obter, listar, `cancelar`).
- **O que entra:** `/vendas` (lista + filtros data/cliente), `/vendas/nova` (cliente, itens com preço/quantidade/desconto/acréscimo), `/vendas/[id]` (detalhe com lucro retornado pelo backend), ação **cancelar venda**.
- **O que fica fora:** Recálculo de custo médio/lucro no frontend, edição de venda, devolução parcial (pós-MVP), emissão fiscal.
- **Dependências:** Produtos, Clientes (prontos) e estoque com saldo (F010/F011/F012). Lucro/custo médio vêm do backend.
- **Prioridade:** Alta.
- **Risco técnico:** Médio (mensagem de "estoque insuficiente" precisa ser bem exposta; venda exige saldo prévio — daí depender de implantação/compras).
- **Ordem sugerida:** 6º.
- **Critérios de aceite:** Venda gera saída de estoque; bloqueio quando saldo insuficiente (mensagem clara); lucro exibido vem do backend; cancelamento funciona; responsivo.
- **Impacto:** `frontend/src/app/vendas/**` (substituir placeholder), `components/vendas/**` (novo), `services/sales.ts` + `hooks/` + `types/` (novos).

F014 — Financeiro: Contas a Receber (frontend)

- **Objetivo:** Gerir recebíveis: criar, listar, registrar pagamentos, ver por cliente, editar e excluir.
- **Escopo exato:** Frontend sobre `ContasReceberController` (completo).
- **O que entra:** `/financeiro/contas-receber` (lista + visão por cliente), criar, registrar pagamento, editar, excluir, detalhe por cliente.
- **O que fica fora:** Contas a pagar (não há entidade), conciliação bancária, juros/multa automáticos.
- **Dependências:** Clientes (pronto). Vendas a prazo idealmente geram recebíveis (verificar acoplamento — atualmente conta a receber é criada manualmente).
- **Prioridade:** Média-alta (gestão).
- **Risco técnico:** Baixo-médio (datas em UTC, como já tratado no controller).

- **Ordem sugerida:** 7º.
- **Critérios de aceite:** CRUD + pagamento funcionais; saldo/valores vêm do backend; histórico preservado (exclusão segue regra do backend); responsivo.
- **Impacto:** `frontend/src/app/financeiro/**` (substituir placeholder), `components/financeiro/**` (novo), `services/receivables.ts` + `hooks/` + `types/` (novos).

F015 — Financeiro: Despesas + Categorias de Despesa (backend + frontend)

- **Objetivo:** Lançar e listar despesas operacionais por categoria.
- **Escopo exato:** Preencher **G2** no backend (endpoint de `CategoriaDespesa`) + frontend de despesas.
- **O que entra:** Backend: `CategoriasDespesaController` (criar/listar/`GET{id}` / `PUT`). Frontend: `/financeiro/despesas` (lista + filtros data/categoria), criar despesa, gerenciar categorias de despesa.
- **O que fica fora:** Update/delete de despesa (G4) salvo se decidido incluir; rateio, centros de custo, recorrência.
- **Dependências:** Backend G2 precede o frontend.
- **Prioridade:** Média-alta (alimenta o dashboard financeiro).
- **Risco técnico:** Baixo (segue padrão MediatR + Fluent API existente).
- **Ordem sugerida:** 8º.
- **Critérios de aceite:** Criar categoria de despesa e despesa via API; controller sem regra; Fluent API; despesa aparece no dashboard financeiro; responsivo; **nova migration apenas se necessário** (entidade já existe).
- **Impacto:** Backend: `Api/Controllers/CategoriasDespesaController.cs` (novo), `Application/Commands|Queries|Handlers|DTOs` (novos), `Inf` Frontend: `app/financeiro/despesas/**`, `components/financeiro/**`, `services/expenses.ts`, `hooks/`, `types/`.

F016 — Dashboard Gerencial e Financeiro (frontend)

- **Objetivo:** Substituir o placeholder da home por um painel real (KPIs, rankings, alertas e gráficos) consumindo os endpoints prontos.
- **Escopo exato:** Frontend sobre `dashboard-gerencial` (consolidado/financeiro/operacional/rankings/alertas/graficos) e `dashboard-financeiro`.
- **O que entra:** Home com KPIs (faturamento, lucro, despesas, recebíveis), rankings (top produtos/clientes), alertas, gráficos (séries), filtros de período (mês/ano/intervalo).
- **O que fica fora:** Cálculo de métricas no frontend (proibido — Princípio VI/VII), exportação, drill-down avançado.
- **Dependências:** Dados gerados por F011/F013/F014/F015 para serem significativos.
- **Prioridade:** Média (importante para gestão; depende de dados reais).
- **Risco técnico:** Médio (biblioteca de gráficos — avaliar leve, ex.: Recharts; justificar no plano conforme Princípio XII/Stack).
- **Ordem sugerida:** 9º.
- **Critérios de aceite:** Todos os números vêm do backend; filtros refazem consultas; responsivo; Dark Only; nenhuma fórmula no cliente.
- **Impacto:** `frontend/src/app/page.tsx` + `components/dashboard/**` (substituir placeholders), `services/dashboard.ts` + `hooks/` + `types/` (novos); possível nova dependência de gráfico (justificar).

F017 — Autenticação e Autorização (backend + frontend)

- **Objetivo:** Proteger o ERP para uso real (login único/poucos usuários da Amani).
- **Escopo exato:** Autenticação JWT no backend + guarda de rotas/sessão no frontend.

- **O que entra:** Entidade/usuário, login (`POST /api/auth/login`), emissão/validação de token, `[Authorize]` nos controllers, tela de login e proteção de rotas no Next.js, logout.
- **O que fica fora (MVP):** Perfis/permissões granulares, recuperação de senha por e-mail, SSO, multi-tenant (pós-MVP).
- **Dependências:** Transversal; idealmente após os fluxos operacionais existirem.
- **Prioridade:** Média-alta (necessária para "entregável real" em produção).
- **Risco técnico:** Médio-alto (toca toda a stack; cuidado para não vazar invariantes; nova migration de usuário).
- **Ordem sugerida:** 10º.
- **Critérios de aceite:** Endpoints protegidos exigem token; login/logout funcionam; segredos fora do código; frontend redireciona não autenticado; build ok.
- **Impacto:** Backend: novo módulo Auth em `Domain/Application/Infra.Data.Api`, `Program.cs` (middleware), migration de usuário. Frontend: `app/login`, middleware de rota, `services/auth.ts`, `api-client.ts` (header Authorization).

F018 — Configurações e Categorias (refinamento, frontend)

- **Objetivo:** Centralizar gestão de apoio: categorias de produto, categorias de despesa e preferências.
- **Escopo exato:** Página Configurações real com CRUD de categorias (endpoints prontos).
- **O que entra:** Gestão de categorias de produto (`/api/categorias`) e de despesa (F015), atalhos para implantação.
- **O que fica fora:** Temas (Dark é fixo), feature flags, integrações.
- **Dependências:** F015 (categorias de despesa).
- **Prioridade:** Baixa-média (refinamento).
- **Risco técnico:** Baixo.
- **Ordem sugerida:** 11º.
- **Critérios de aceite:** CRUD de categorias funcional; responsivo; sem regra no cliente.
- **Impacto:** `frontend/src/app/configuracoes/**` (substituir placeholder), `components/configuracoes/**`, `services/categories.ts` (estender).

3. Roadmap recomendado por fases

Fase 1 — Essencial para usar o sistema (operação ponta a ponta)

Objetivo: a Amani consegue lançar dados iniciais, comprar, receber, ver estoque e vender.

1. **F008** Consulta de Estoque (backend)
2. **F009** Fornecedores (frontend)
3. **F010** Implantação Inicial (frontend)
4. **F011** Compras e Recebimentos (frontend)
5. **F012** Estoque (frontend)
6. **F013** Vendas (frontend)

*Ao fim da Fase 1 já existe um **MVP operacional**: cadastra, semeia estoque/caixa, compra → recebe → estoca → vende com validação de saldo e lucro.*

Fase 2 — Importante para gestão

1. **F014** Contas a Receber (frontend)
2. **F015** Despesas + Categorias de Despesa (backend G2 + frontend)

3. **F016** Dashboard Gerencial/Financeiro (frontend)
4. **F017** Autenticação e Autorização (full stack)

Ao fim da Fase 2 o sistema está **pronto para uso real em produção** com controle financeiro, visão gerencial e acesso protegido.

Fase 3 — Refinamento

1. **F018** Configurações e Categorias
 - Detalhe de Produto exibindo saldo + custo médio (depende de F008)
 - Paginação e busca server-side nas listas (hoje busca é local)
 - Melhor leitura de `{ error }` no `api-client.ts`
 - Update/delete de Despesas e inativação de Produto/Fornecedor (G4)

Fase 4 — Futuro pós-MVP

- Relatórios exportáveis (PDF/Excel) e drill-down
- Contas a pagar / fluxo de caixa projetado
- Perfis e permissões granulares; multi-usuário avançado
- Integrações com marketplaces e emissão fiscal
- Devolução de venda, multi-depósito, estoque mínimo com alertas automáticos
- PWA/offline para operação em viagem (reforça Princípio VIII Mobile First)

Observações de conformidade

- Todas as features de Fase 1-2 **reaproveitam endpoints existentes**; as únicas adições de backend são **F008** (consulta de estoque) e **G2 em F015** (categorias de despesa) — ambas dentro de Clean Architecture/DDD Lite/Fluent API.
- **Estoque continua por movimentações** em todas as propostas: F008 só lê e agrega; F011/F013 geram movimentações; nenhuma introduz campo fixo de saldo.
- Frontend permanece sem regra crítica de negócio (lucro, custo médio, métricas e validações vêm do backend), respeitando os Princípios VI e VII.